

Day 6: Collections Framework – Part 1 (15-June-2026, Monday)

Session Time: 9:00 AM – 4:30 PM (Lunch Break: 12:30 PM – 1:30 PM, Tea Breaks: 10:30 AM & 3:00 PM)

Session Objectives

- By the end of Day 6, learners will be able to:
- Explain the benefits of the Java Collections Framework over arrays.
 - Use `ArrayList` for dynamic array operations and understand its internal resizing.
 - Differentiate between `ArrayList`, `LinkedList`, and `Vector` in terms of performance and thread-safety.
 - Implement `Stack` (LIFO) and `Queue` (FIFO) using `Deque` and `LinkedList`.
 - Traverse collections using `Iterator` and `ListIterator`.
 - Solve LeetCode: *Implement Queue using Stacks* and revisit *Valid Parentheses* using stack.
 - Solve HackerRank: *Java List* and *Java Stack*.
 - Design an **Inventory & Warehouse Tracking System** using appropriate collections.

Detailed Agenda (Time Slots & Activities)

Time Slot	Activity	Duration
9:00 – 9:15	Recap of Day 5 & Day 6 Overview – Why Collections?	15 min
9:15 – 10:00	<code>ArrayList</code> – Creation, Methods, Resizing, Performance	45 min
10:00 – 10:30	<code>LinkedList</code> – Node Structure, Methods, When to Use	30 min
10:30 – 10:45	Tea Break	15 min
10:45 – 11:15	<code>Vector</code> – Legacy Class, Differences from <code>ArrayList</code>	30 min
11:15 – 11:45	<code>Stack</code> Class – LIFO Operations, Peek, Pop, Push	30 min
11:45 – 12:15	<code>Queue</code> Interface – <code>Deque</code> , <code>ArrayDeque</code> , <code>PriorityQueue</code> basics	30 min
12:15 – 12:30	Hands-on: Comparing <code>ArrayList</code> vs <code>LinkedList</code> Performance	15 min
12:30 – 1:30	Lunch Break	60 min

1:30 – 2:00	Iterators – Iterator vs ListIterator, Fail-Fast Behavior	30 min
2:00 – 2:30	LeetCode: Implement Queue using Stacks (Approach)	30 min
2:30 – 2:45	HackerRank: Java List & Java Stack Walkthrough	15 min
2:45 – 3:00	Tea Break	15 min
3:00 – 3:30	LeetCode: Valid Parentheses (Stack-based) Revisited	30 min
3:30 – 4:00	Corporate Use Case: Inventory & Warehouse Tracking System	30 min
4:00 – 4:30	Hands-on coding exercise, Q&A, Homework	30 min

1. Java Collections Framework – Overview

1.1 Limitations of Arrays

- Fixed size – cannot grow or shrink dynamically.
- Only store homogeneous types (though covariance allows some flexibility).
- No built-in methods for common operations (sort, search, reverse).
- Manual resizing leads to verbose code.

1.2 What is Collections Framework?

- A unified architecture for storing, retrieving, and manipulating groups of objects.
- Includes: **Interfaces** (List, Set, Queue, Deque, Map), **Implementations** (ArrayList, HashSet, etc.), **Algorithms** (sort, binarySearch, reverse).
- Part of `java.util` package.

1.3 Core Collection Interfaces Hierarchy (simplified)

```

Iterable (interface)
├── Collection (interface)
│   ├── List (interface) – ordered, allows duplicates, index-based
│   │   ├── ArrayList
│   │   ├── LinkedList
│   │   └── Vector (legacy)
│   ├── Set (interface) – no duplicates, no order (will cover Day 7)
│   └── Queue (interface) – FIFO, priority
│       └── Deque (interface) – double-ended queue

```

2. ArrayList

2.1 Definition

- Resizable-array implementation of the `List` interface.
- Maintains insertion order.
- Allows duplicates and `null` values.
- Provides **random access** (index-based) in $O(1)$ time.

2.2 Creating an ArrayList

```
// Default initial capacity = 10
ArrayList<String> list = new ArrayList<>();

// With initial capacity
ArrayList<Integer> numbers = new ArrayList<>(20);

// From another collection
ArrayList<String> copy = new ArrayList<>(existingList);
```

2.3 Common Methods

Method	Description	Time Complexity
<code>add(E e)</code>	Appends to end	$O(1)$ amortized
<code>add(int index, E e)</code>	Inserts at position	$O(n)$
<code>get(int index)</code>	Returns element	$O(1)$
<code>set(int index, E e)</code>	Replaces element	$O(1)$
<code>remove(int index)</code>	Removes by index	$O(n)$
<code>remove(Object o)</code>	Removes first occurrence	$O(n)$
<code>indexOf(Object o)</code>	Returns index or -1	$O(n)$
<code>contains(Object o)</code>	Checks existence	$O(n)$
<code>size()</code>	Returns number of elements	$O(1)$
<code>isEmpty()</code>	Checks if empty	$O(1)$
<code>clear()</code>	Removes all elements	$O(n)$

2.4 Internal Resizing Mechanism

- When capacity is exceeded, `ArrayList` creates a new array of size $(oldCapacity * 1.5)$ (approx).

- Copies old elements using `System.arraycopy()` .
- Resizing is expensive; use `ensureCapacity()` if you know size in advance.

2.5 When to Use ArrayList?

- Frequent random access (get/set).
- Adding/removing mostly at the end.
- Iteration is fast (cache-friendly).

3. LinkedList

3.1 Definition

- Doubly-linked list implementation of `List` and `Deque` interfaces.
- Each element (node) contains data and pointers to prev/next nodes.
- Does **not** implement random access efficiently.

3.2 Creating a LinkedList

```
LinkedList<String> linkedList = new LinkedList<>();
```

3.3 Common Methods (beyond List interface)

Method	Description
<code>addFirst(E e)</code>	Inserts at beginning
<code>addLast(E e)</code>	Inserts at end
<code>removeFirst()</code>	Removes first element
<code>removeLast()</code>	Removes last element
<code>getFirst()</code>	Returns first element
<code>getLast()</code>	Returns last element
<code>offer(E e)</code>	Adds to end (Queue method)
<code>poll()</code>	Removes and returns head (Queue)
<code>peek()</code>	Returns head without removal

3.4 Time Complexities

Operation	ArrayList	LinkedList
get(index)	O(1)	O(n)
add(index)	O(n)	O(n) (but faster if near ends)
add at end	O(1) amortized	O(1)
remove at end	O(1)	O(1)
remove at beginning	O(n)	O(1)
iterate	O(n)	O(n) (similar)

3.5 When to Use LinkedList?

- Frequent insertions/deletions at **beginning or middle** (but need to traverse to position).
- Implementing Queue/Deque (prefer `ArrayDeque` for performance though).
- Memory fragmentation not a concern.

4. Vector (Legacy)

4.1 Definition

- Synchronized (thread-safe) version of `ArrayList` (before Java 1.2).
- Methods like `addElement()`, `elementAt()`, `removeElement()` – similar to `ArrayList`.
- Default capacity 10, growth factor 100% (doubling).

4.2 Why Avoid Vector?

- Synchronization overhead even when not needed (slower).
- Legacy methods are verbose.
- `Collections.synchronizedList(new ArrayList<>())` gives better control.

4.3 When to Use Vector?

- Rarely in new code. Only when working with legacy APIs.

5. Stack Class

5.1 Definition

- Subclass of `Vector` – LIFO (Last-In-First-Out) data structure.

- Methods:
 - `push(E item)` – adds to top.
 - `pop()` – removes and returns top.
 - `peek()` – returns top without removal.
 - `empty()` – checks if empty.
 - `search(Object o)` – returns 1-based position from top.

5.2 Problems with Stack Class

- Extends `Vector` – inherits all `Vector` methods (e.g., `add(int, E)`) allowing violation of LIFO.
- Prefer `Deque` interface with `ArrayDeque` for stack operations.

5.3 Modern Stack Implementation

```
Deque<String> stack = new ArrayDeque<>();
stack.push("first");
String top = stack.pop();
```

6. Queue Interface

6.1 Definition

- FIFO (First-In-First-Out) data structure.
- Common implementations: `LinkedList`, `ArrayDeque`, `PriorityQueue`.

6.2 Queue Methods – Two Styles

Throws Exception	Returns Special Value	Description
<code>add(e)</code>	<code>offer(e)</code>	Insert element (false if full)
<code>remove()</code>	<code>poll()</code>	Remove & return head (null if empty)
<code>element()</code>	<code>peek()</code>	Return head without removal (null if empty)

6.3 Deque (Double-Ended Queue)

- Supports insertion/removal at both ends.
- Methods: `addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`, `offerFirst()`, `offerLast()`, etc.
- Preferred for stack and queue.

6.4 ArrayDeque

- Resizable array implementation of `Deque`.
- Faster than `LinkedList` for most operations (no node overhead).
- Not thread-safe.

6.5 PriorityQueue

- Elements ordered by natural ordering or comparator (not FIFO).
- Heap-based – `offer` / `poll` $O(\log n)$.
- Does not allow `null`.

7. Iterators

7.1 Iterator Interface

- Provides uniform way to traverse any `Collection`.
- Methods: `hasNext()`, `next()`, `remove()` (optional).

7.2 ListIterator

- Extends `Iterator` – works only for `List`.
- Additional methods: `hasPrevious()`, `previous()`, `nextIndex()`, `previousIndex()`, `set(E e)`, `add(E e)`.

7.3 Enhanced For-Loop (for-each)

- Syntactic sugar – uses iterator behind the scenes.
- Cannot remove elements (except using iterator's `remove()`).

7.4 Fail-Fast vs Fail-Safe

Type	Behavior	Example
Fail-Fast	Throws <code>ConcurrentModificationException</code> if collection modified during iteration (except via iterator's own <code>remove</code>).	<code>ArrayList</code> , <code>HashSet</code>
Fail-Safe	Works on a copy, no exception but may not reflect latest changes.	<code>CopyOnWriteArrayList</code>

8. LeetCode Problems – Approach Discussion

8.1 Implement Queue using Stacks

- **Problem:** Implement a FIFO queue using two LIFO stacks.
- **Approach:**
 - Use `inStack` for pushes, `outStack` for pops/peeks.
 - When `outStack` is empty, pop all from `inStack` and push to `outStack` (reverses order).
 - Each element moves at most twice → $O(1)$ amortized per operation.
- **Complexities:** push $O(1)$, pop amortized $O(1)$.
- **Implementation choice:** `Deque` as stack (`ArrayDeque`) or `Stack` class.

8.2 Valid Parentheses (Revisited)

- Already covered Day 4, but reinforce stack usage:
 - Push opening brackets.
 - On closing bracket, pop top – must match.
 - Use `HashMap` for bracket pairs.
 - Edge: stack empty on closing → invalid.
 - **Why revisit?** Stack is a core collection; this problem solidifies understanding.
-

9. Corporate Use Case: Inventory & Warehouse Tracking System

9.1 Problem Context

A warehouse management system needs to track:

- **Inventory items** (SKU, name, quantity, location).
- **Stock movements** – items arrive (add to inventory) and are picked (removed).
- **Undo/Redo operations** for accidental picks or restocks (requires stack).
- **Order processing queue** – FIFO for shipping orders.
- **Recent restock history** – LIFO for viewing last 5 actions.
- **Bulk operations** – adding multiple items at once.

9.2 Application of Day 6 Topics

Requirement	Java Feature Used
Store current inventory (dynamic)	<code>ArrayList<InventoryItem></code> – random access by index

list of items)	for updates
Add new item to inventory	<code>ArrayList.add()</code>
Remove item by SKU	Iterate with <code>Iterator</code> to safely remove
Order processing (FIFO)	<code>Queue<Order></code> using <code>ArrayDeque</code> or <code>LinkedList</code>
Undo last inventory action (e.g., accidental removal)	<code>Deque<Action></code> as stack – push each action, pop on undo
Redo after undo	Second stack (<code>redoStack</code>)
Recent restock history (last 5 pushes)	<code>Stack</code> or <code>ArrayDeque</code> with size limit
Batch insert of items (faster than repeated adds)	<code>ArrayList.addAll(collection)</code>
Traverse inventory to generate report	<code>Iterator</code> or for-each
Modify inventory while iterating (e.g., apply discount)	<code>ListIterator</code> for bidirectional traversal and modification
Thread-safe inventory for concurrent access (advanced)	Not needed now, but mention <code>Vector</code> (legacy) or <code>Collections.synchronizedList</code>

9.3 Example Data Structures

```
// Main inventory
ArrayList<InventoryItem> inventory = new ArrayList<>();

// Order queue
Queue<Order> shippingQueue = new ArrayDeque<>();

// Undo/Redo stacks
Deque<InventoryAction> undoStack = new ArrayDeque<>();
Deque<InventoryAction> redoStack = new ArrayDeque<>();

// Restock history (stack)
Deque<RestockEvent> restockHistory = new ArrayDeque<>();
```

9.4 Business Logic Examples

• Add item:

- `inventory.add(newItem);`
- `undoStack.push(new AddAction(newItem));`
- `restockHistory.push(new RestockEvent(newItem.getSku(), newItem.getQuantity()));`

- **Remove item (pick for order):**
 - Find item using loop + iterator.
 - `iterator.remove();`
 - `undoStack.push(new RemoveAction(item));`
- **Process order queue:**
 - `Order next = shippingQueue.poll();`
 - Remove corresponding items from inventory.
- **Undo:**
 - `InventoryAction last = undoStack.pop();`
 - `last.undo();` (re-add removed item or remove added item)
 - `redoStack.push(last);`

9.5 Discussion Questions

- Why use `ArrayList` for inventory rather than `LinkedList`? (Random access by index when updating quantities; iteration speed similar.)
- Would `Vector` be better for multi-threaded warehouse system? (Yes, but modern approach uses `ConcurrentHashMap` or synchronized wrappers.)
- How to limit `restockHistory` to only last 5 items? (Check size before push; if ≥ 5 , remove last element using `removeLast()` on `ArrayDeque` used as stack.)
- What if we need to search inventory by SKU quickly? (Use `HashMap` – Day 7).

10. Hands-on Exercises (Without Code Lines)

10.1 ArrayList Resizing Simulation

Describe what happens internally when:

- `ArrayList<Integer> list = new ArrayList<>(3);`
- `list.add(10); list.add(20); list.add(30);` (capacity now 3)
- `list.add(40);` – explain new capacity (likely 4 or 5? Oracle grows by 1.5x, so 3→4? Actually formula: $\text{oldCapacity} + (\text{oldCapacity} >> 1) = 3 + 1 = 4$)

10.2 LinkedList vs ArrayList – Choose Wisely

For each scenario, pick better one:

- Reading 100,000 elements by index in a loop. (ArrayList)
- Removing first element of a large list repeatedly. (LinkedList)
- Adding elements only at the end, never removing. (ArrayList)
- Implementing a music playlist with next/previous track. (LinkedList – doubly linked)

• implementing a music playlist with next/previous track. (LinkedList = doubly linked)

10.3 Stack Simulation

Given push sequence: A, B, C, pop, D, pop, pop. Write final stack contents and print order of popped elements.

10.4 Iterator Removal

Explain why this code fails (conceptually):

```
List<String> list = new ArrayList<>();
// ... add elements
for (String s : list) {
    if (s.equals("bad")) list.remove(s);
}
```

What is the fix? (Use `iterator.remove()` or collect and remove later.)

11.5 Implement Queue using Stacks – Dry Run

Show steps for: push(1), push(2), pop(), push(3), pop(), pop(). Trace inStack/outStack contents after each operation.

11. Quiz & Quick Check (Conceptual)

1. Which of the following provides O(1) random access?

- (a) LinkedList
- (b) ArrayList
- (c) Vector
- (d) Both b and c

2. Which method of Queue returns null if empty instead of throwing exception?

- (a) `remove()`
- (b) `element()`
- (c) `poll()`
- (d) `add()`

3. What does `ListIterator` provide that `Iterator` does not?

- (a) Ability to remove elements
- (b) Ability to traverse backwards

- (c) Ability to check hasNext()
- (d) Ability to convert to array

4. Which class is legacy and synchronized?

- (a) ArrayList
- (b) LinkedList
- (c) Vector
- (d) ArrayDeque

5. In the "Implement Queue using Stacks" problem, what is the time complexity of pop operation in amortized sense?

- (a) $O(n)$
- (b) $O(1)$
- (c) $O(\log n)$
- (d) $O(n^2)$

Answers: 1-d (both ArrayList and Vector have $O(1)$ get), 2-c, 3-b, 4-c, 5-b

12. Additional Resources & Cheat Sheets

- **ArrayList vs LinkedList performance** – concrete numbers for different operations.
 - **Deque as Stack vs Stack class** – why prefer Deque.
 - **Iterator fail-fast** – how `modCount` works.
 - **Queue method pairs:** add/offer, remove/poll, element/peek.
-

13. Closing Summary

Day 6 introduced the Java Collections Framework, focusing on `List` implementations (`ArrayList`, `LinkedList`, `Vector`), `Stack`, `Queue`, and `Iterators`. Learners understood when to use each collection based on performance characteristics. The corporate case study – Inventory & Warehouse Tracking – demonstrated real-world applications: `ArrayList` for dynamic inventory, `Queue` for order processing, `Stack` for undo/redo and history. LeetCode problems reinforced stack and queue usage, while HackerRank challenges solidified syntax.

Key Takeaways:

- `ArrayList` is the default choice for dynamic arrays; `LinkedList` is better for frequent front insertions/deletions.
- `Stack` extends `Vector`; prefer `Deque` interface with `ArrayDeque`.

- Queue implementations: `LinkedList`, `ArrayDeque`, `PriorityQueue`.
- `Iterator` allows safe removal during traversal.
- Collections make data structure handling concise and powerful.

End of Day 6